EE-6250 VLSI Testing
Homework #3 Option 2 Report

Built-In Speed Grading for an Unsigned Adder

CS 110070016 蕭以勝

1. Assignment Interpretation and Implementation Summary

This report implements Option 2 of Homework #3. The design contains a pure unsigned 6-bit adder for part (a), and a BISTed adder for parts (b)-(d). The BISTed adder uses a 4-stage LFSR as the automatic pattern generator and a MISR as the signature analyzer. For part (e), the circuit under test is replaced by an unsigned 12-bit adder and the maximum-speed search procedure is repeated.

Important implementation choice: the 4-stage LFSR produces 15 non-zero states. Since a 6-bit adder has two 6-bit operands, each 4-bit state is expanded through a deterministic combinational mapping into a full 12-bit input vector for A[5:0] and B[5:0]. The same idea is used for the 12-bit case, where the LFSR state is expanded into two 12-bit operands.

Item	6-bit BISTed adder	12-bit BISTed adder
CUT	Unsigned 6-bit ripple-carry adder	Unsigned 12-bit ripple-carry adder
LFSR	4-stage LFSR, initial state = 0001	4-stage LFSR, initial state = 0001
Number of BIST patterns	15	15
MISR width	7 bits	13 bits
Golden signature	0x4d	0x458
DC synthesis constraint	same as swept period	same as swept period
Final speed result	800 ps = 1250.000 MHz	1000 ps = 1000.000 MHz

2. Part (a): Unsigned 6-bit Adder Logic Circuit

The unsigned 6-bit adder is implemented as a ripple-carry adder. It consists of six full-adder stages. The carry-out of bit i is connected to the carry-in of bit i+1. Since the maximum sum is 63 + 63 = 126, the output requires 7 bits.

For each bit position i, the logic equations are:

p_i = a_i XOR b_i

g_i = a_i AND b_i

sum_i = p_i XOR c_i

c_{i+1} = g_i OR (p_i AND c_i), with c_0 = 0 and sum[6] = c_6

The following pure-adder waveform verifies the 6-bit adder without any LFSR or MISR. The tested cases include no-carry, simple-carry, long carry-propagation, and maximum-input cases.

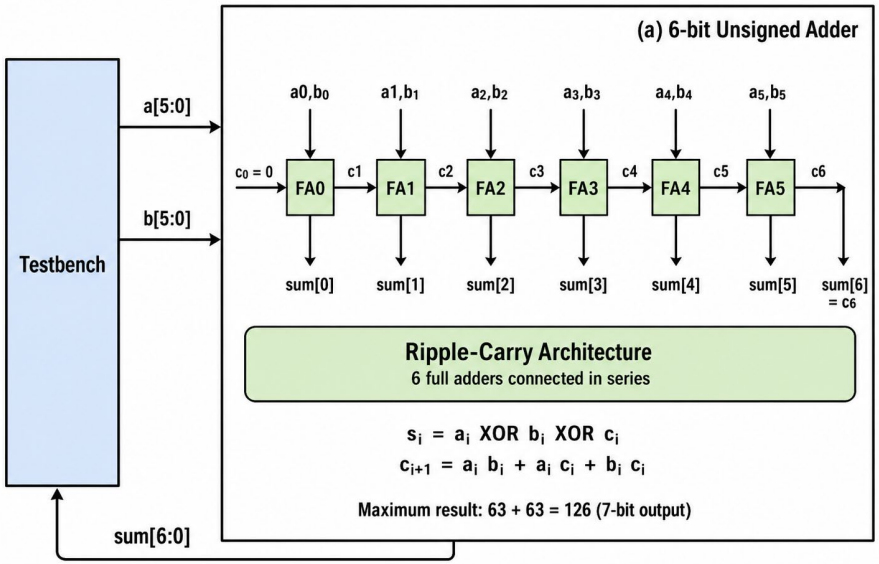


Figure 1. 6-bit adder architecture

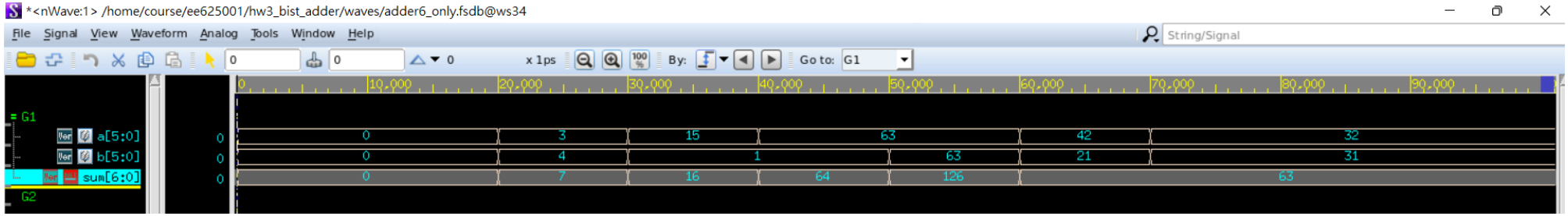


Figure 2. Pure 6-bit unsigned adder waveform. Example checks include 63 + 1 = 64 and 63 + 63 = 126.

3. Part (b): BISTed Adder Architecture and Golden Signature

The BISTed design contains a 4-stage LFSR automatic pattern generator, a combinational input-expansion network, the unsigned adder as the circuit under test, and a MISR signature analyzer.

One BIST pattern is applied per clock cycle. After 15 patterns, the final MISR state is used as the fault-free golden signature.

Architecture: 4-stage LFSR -> input expansion network -> unsigned adder CUT -> MISR -> final signature.

LFSR recurrence: feedback = lfsr_state[3] XOR lfsr_state[0], and next_state = {lfsr_state[2:0], feedback}. Starting from 0001, the LFSR visits 15 non-zero states before repetition.

For the 6-bit case, the operand expansion is: A = {lfsr_state, lfsr_state[3:2]} and B = {lfsr_state[1:0], lfsr_state}. For the 12-bit case, a deterministic expansion network in the Verilog code generates A[11:0] and B[11:0].

MISR update rule: signature_next = (signature << 1) XOR tap_correction XOR sum. The 6-bit adder uses a 7-bit MISR polynomial x^7 + x + 1, and the 12-bit adder uses a 13-bit MISR polynomial x^13 + x^4 + x^3 + x + 1.

Design	MISR width	Polynomial / tap mask	Fault-free golden signature
6-bit adder	7	x^7 + x + 1 / 7'b0000011	0x4d
12-bit adder	13	x^13 + x^4 + x^3 + x + 1 / 13'b0000000011011	0x458

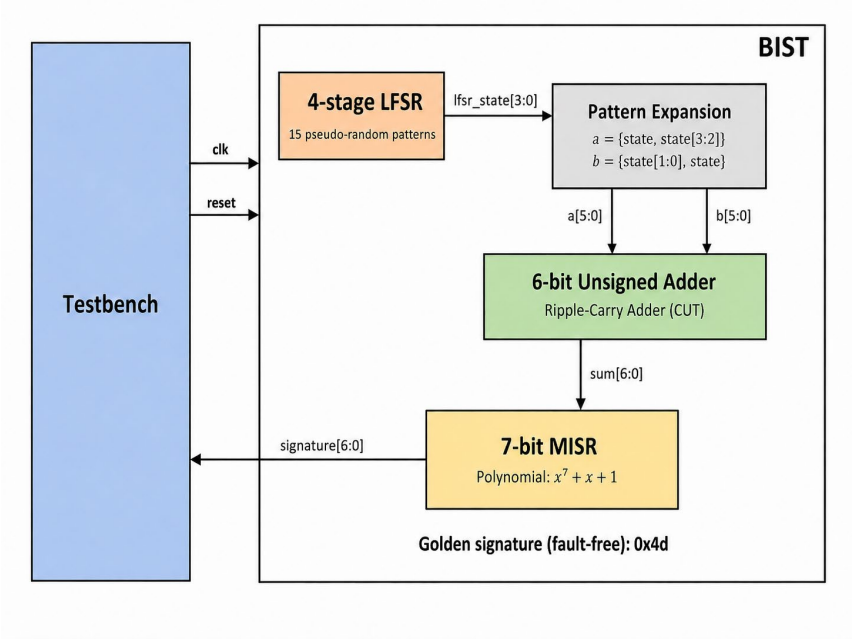


Figure 3. BIST architecture

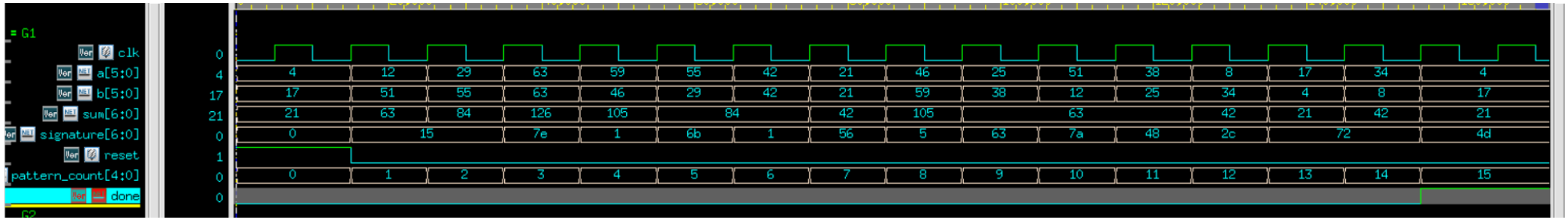


Figure 4. 6-bit BISTed adder RTL simulation at 100 MHz. The final signature is 0x4d.

4. Part (c): 100 MHz Simulation Environment and Waveforms

A 100 MHz clock corresponds to a 10 ns clock period, or 10000 ps in the testbench. The testbench accepts PERIOD_PS as a plusarg, generates the clock, releases reset on a negative clock edge to avoid reset recovery/removal violations in gate-level simulation, runs the BIST sequence until done is asserted, and compares the final MISR signature with the golden signature.

The important waveform signals are clk, reset, lfsr_state, a, b, sum, signature, pattern_count, and done. These signals show the clock period, LFSR pattern progression, adder input/output responses, MISR compaction, and completion of the BIST sequence.

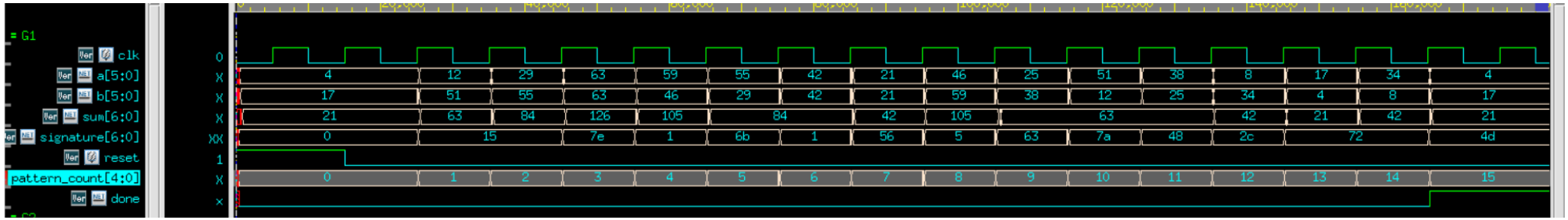


Figure 5. 6-bit BISTed adder gate-level SDF simulation at 100 MHz. The final signature remains 0x4d, matching the golden signature.

5. Part (d): Maximum Operating Speed Procedure

For the maximum-speed search, I used a same-clock synthesis-and-simulation sweep. For each candidate clock period, the Design Compiler constraint is set to the same period that is later used in gate-level SDF simulation. Therefore, the synthesized netlist and SDF are regenerated for each clock candidate instead of keeping a fixed 10.0 ns implementation during the sweep.

This procedure measures whether the BISTed design can be synthesized and simulated successfully at a given candidate clock. A period is accepted only when synthesis generates the corresponding gate-level netlist and SDF file, gate-level simulation finishes normally, the final MISR signature matches the golden signature, HW3_PASS is printed, and no timing violation is reported in the simulation log.

1. Choose a candidate clock period P in ps.
2. Convert P to a Design Compiler clock constraint in ns by using CLK_PERIOD_NS = P / 1000.
3. Synthesize the BISTed adder using this same-period clock constraint, and generate a new gate-level Verilog netlist and SDF file.
4. Run gate-level SDF simulation with +PERIOD_PS=P, so the simulation clock matches the synthesis constraint.
5. Decrease the candidate clock period in 100 ps steps. The sweep stops at the first failing period after at least one passing period has been found.
6. The smallest passing period is reported as the maximum operating speed using Fmax(MHz) = 1,000,000 / period_ps.

The sweep script implements this procedure and also saves the fastest passing FSDB waveform and the first failing FSDB waveform for later inspection. In the final sweep, the 6-bit BISTed adder passed at 800 ps and failed at 700 ps, while the 12-bit BISTed adder passed at 1000 ps and failed at 900 ps.

Design	Best passing period	First failing period checked	Maximum speed
6-bit BISTed adder	800 ps	700 ps	1250.000 MHz
12-bit BISTed adder	1000 ps	900 ps	1000.000 MHz

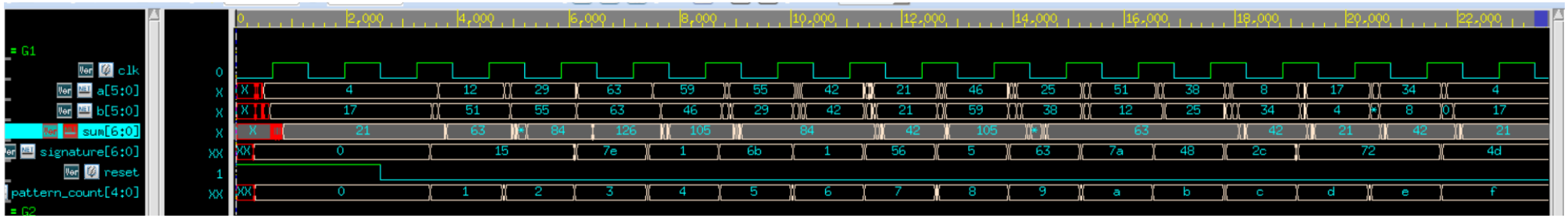


Figure 6. 6-bit gate-level BIST waveform at the best passing period, 800 ps. The final signature is 0x4d.

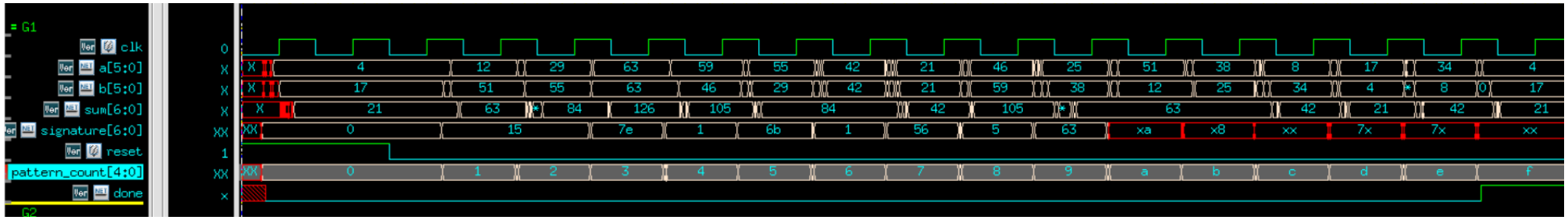


Figure 7. 6-bit gate-level BIST waveform at 700 ps. The simulation fails, so 700 ps is not accepted.

6. Part (e): Replacing the CUT with a 12-bit Adder

The same BIST and speed-grading procedure is repeated after replacing the circuit under test with an unsigned 12-bit ripple-carry adder. The 4-stage LFSR still supplies 15 patterns through deterministic input expansion, and the output responses are compacted by a 13-bit MISR. The fault-free golden signature is 0x458.

Using the same synthesis-and-simulation sweep and the same 100 ps resolution, the smallest passing period for the 12-bit BISTed adder is 1000 ps. Therefore, the maximum operating speed is 1,000,000 / 1000 = 1000.000 MHz. The next faster candidate, 900 ps, fails in gate-level simulation and is not accepted.

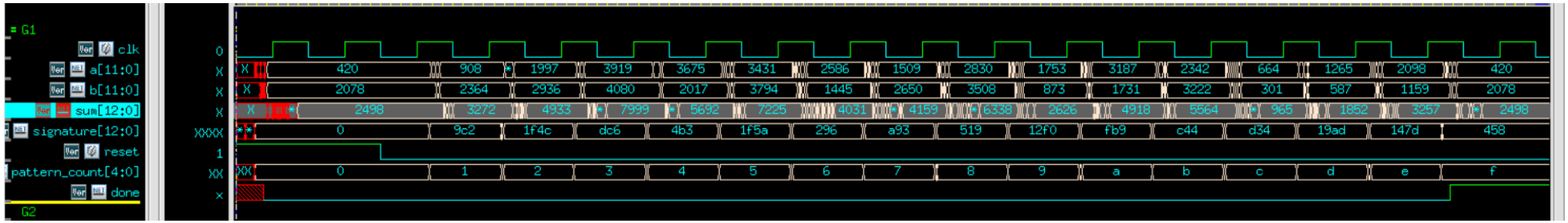


Figure 8. 12-bit gate-level BIST waveform at the best passing period, 1000 ps. The final signature is 0x458.

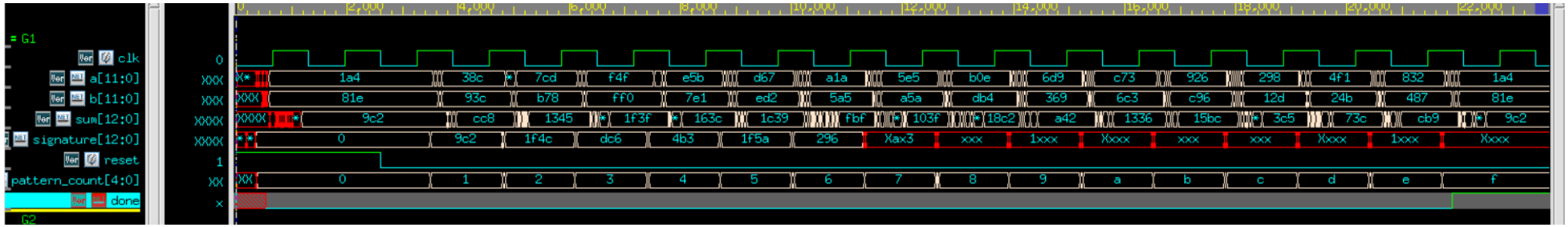


Figure 9. 12-bit gate-level BIST waveform at 900 ps. The simulation fails, so 900 ps is not accepted.

7. Final Results

Item	6-bit BISTed adder	12-bit BISTed adder
Golden signature	0x4d	0x458
100 MHz gate-level result	PASS	Not required by part (e); final speed result reported
Synthesis / simulation clock relation	Same clock period for each candidate	Same clock period for each candidate
Sweep resolution	100 ps	100 ps
First failing period checked	700 ps	900 ps
Smallest passing period	800 ps	1000 ps
Maximum operating speed	1250.000 MHz	1000.000 MHz

The 12-bit maximum speed is lower than the 6-bit maximum speed because the ripple-carry adder has a longer carry-propagation path. Both designs use the same style of LFSR-based pattern generation and MISR response compaction, but the 12-bit CUT and the 13-bit MISR produce longer timing paths than the 6-bit case.

Appendix A. Verilog Source Code

A.1 bist_adder.v

```
// ------------------------------------------------------------------------------
// EE-6250 VLSI Testing HW3
// BISTed unsigned adder using a 4-stage LFSR APG and a MISR signature analyzer.
// Supported top modules:
//   - bist_adder6  : unsigned 6-bit adder,  7-bit MISR signature
//   - bist_adder12 : unsigned 12-bit adder, 13-bit MISR signature
// ------------------------------------------------------------------------------
`timescale 1ns/1ps

module bist_adder6 (
    input  wire       clk,
    input  wire       reset,
    output wire [3:0] lfsr_state,
    output wire [5:0] a,
    output wire [5:0] b,
    output wire [6:0] sum,
    output wire [6:0] signature,
    output wire [4:0] pattern_count,
    output wire       done
);
    bist_adder #(.N(6)) u_bist_adder (
        .clk(clk),
        .reset(reset),
        .lfsr_state(lfsr_state),
        .a(a),
        .b(b),
        .sum(sum),
        .signature(signature),
        .pattern_count(pattern_count),
        .done(done)
    );
endmodule

module bist_adder12 (
    input  wire        clk,
    input  wire        reset,
    output wire [3:0]  lfsr_state,
    output wire [11:0] a,
    output wire [11:0] b,
    output wire [12:0] sum,
    output wire [12:0] signature,
    output wire [4:0]  pattern_count,
    output wire        done
);
    bist_adder #(.N(12)) u_bist_adder (
        .clk(clk),
        .reset(reset),
        .lfsr_state(lfsr_state),
        .a(a),
        .b(b),
        .sum(sum),
        .signature(signature),
        .pattern_count(pattern_count),
        .done(done)
    );
endmodule
```

```
// Generic BIST top. N should be 6 or 12 for this assignment.
module bist_adder #(
    parameter N = 6
)(
    input  wire         clk,
    input  wire         reset,
    output reg  [3:0]   lfsr_state,
    output wire [N-1:0] a,
    output wire [N-1:0] b,
    output wire [N:0]   sum,
    output reg  [N:0]   signature,
    output reg  [4:0]   pattern_count,
    output reg          done
);
    localparam SUM_W = N + 1;

    // 4-stage LFSR with primitive polynomial x^4 + x + 1.
    // Starting from 0001, this generates 15 nonzero states.
    wire lfsr_feedback;
    wire [3:0] lfsr_next;
    assign lfsr_feedback = lfsr_state[3] ^ lfsr_state[0];
    assign lfsr_next     = {lfsr_state[2:0], lfsr_feedback};

    // Expand the 4-bit LFSR state into two adder operands.
    // This keeps exactly 15 BIST patterns while still driving all adder inputs.
    generate
        if (N == 6) begin : GEN_PATTERN_6
            assign a = {lfsr_state, lfsr_state[3:2]};
            assign b = {lfsr_state[1:0], lfsr_state};
        end else begin : GEN_PATTERN_12
            assign a = {lfsr_state,
                        (lfsr_state ^ 4'b1011),
                        {lfsr_state[1:0], lfsr_state[3:2]}};
            assign b = {{lfsr_state[0], lfsr_state[3:1]},
                        lfsr_state,
                        ~lfsr_state};
        end
    endgenerate

    // Unsigned ripple-carry adder, expressed as generate logic equations.
    ripple_adder #(.N(N)) u_adder (
        .a(a),
        .b(b),
        .sum(sum)
    );

    // MISR masks. The LSB is the constant term of the polynomial.
    // For 6-bit adder:  W=7,  p(x)=x^7+x+1.
    // For 12-bit adder: W=13, p(x)=x^13+x^4+x^3+x+1.
    wire [SUM_W-1:0] tap_mask;
    assign tap_mask = (N == 6) ? 7'b0000011 : 13'b0000000011011;

    wire feedback_misr;
    wire [SUM_W-1:0] sig_shifted;
    wire [SUM_W-1:0] sig_next;
    assign feedback_misr = signature[SUM_W-1];
    assign sig_shifted   = {signature[SUM_W-2:0], 1'b0};
    assign sig_next      = sig_shifted ^ (feedback_misr ? tap_mask : {SUM_W{1'b0}}) ^ sum;

    always @(posedge clk or posedge reset) begin
```

```
        if (reset) begin
            lfsr_state    <= 4'b0001;
            pattern_count <= 5'd0;
            signature     <= {SUM_W{1'b0}};
            done          <= 1'b0;
        end else if (!done) begin
            // Compact the CUT response generated by the current LFSR state.
            signature <= sig_next;

            // Move to the next pseudo-random pattern.
            lfsr_state <= lfsr_next;

            if (pattern_count == 5'd14) begin
                pattern_count <= 5'd15;
                done          <= 1'b1;
            end else begin
                pattern_count <= pattern_count + 5'd1;
                done          <= 1'b0;
            end
        end
    end
endmodule

module ripple_adder #(
    parameter N = 6
)(
    input  wire [N-1:0] a,
    input  wire [N-1:0] b,
    output wire [N:0]   sum
);
    wire [N:0] carry;
    wire [N-1:0] p;
    wire [N-1:0] g;

    assign carry[0] = 1'b0;
    assign p = a ^ b;
    assign g = a & b;

    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : GEN_FULL_ADDER
            assign sum[i] = p[i] ^ carry[i];
            assign carry[i+1] = g[i] | (p[i] & carry[i]);
        end
    endgenerate

    assign sum[N] = carry[N];
endmodule
```

Appendix B. Testbenches

B.1 tb_adder6.v - pure 6-bit adder testbench

```
`timescale 1ns/1ps

module tb_adder6_only;

    reg  [5:0] a;
    reg  [5:0] b;
    wire [6:0] sum;

    // Pure 6-bit unsigned adder only
    ripple_adder #(.N(6)) dut (
        .a(a),
        .b(b),
        .sum(sum)
    );

    task apply_test;
        input [5:0] ta;
        input [5:0] tb;
        input [6:0] expected;
        begin
            a = ta;
            b = tb;
            #10;

            if (sum === expected)
                $display("PASS: a=%b b=%b sum=%b expected=%b", a, b, sum, expected);
            else
                $display("FAIL: a=%b b=%b sum=%b expected=%b", a, b, sum, expected);
        end
    endtask

    initial begin
`ifdef FSDB
        $fsdbDumpfile("waves/adder6_only.fsdb");
        $fsdbDumpvars(0, tb_adder6_only);
`else
        $dumpfile("waves/adder6_only.vcd");
        $dumpvars(0, tb_adder6_only);
`endif
    end

    initial begin
        a = 6'b000000;
        b = 6'b000000;

        #10;

        // Test 1: zero + zero
        apply_test(6'b000000, 6'b000000, 7'b0000000);  // 0 + 0 = 0

        // Test 2: no carry
        apply_test(6'b000011, 6'b000100, 7'b0000111);  // 3 + 4 = 7
```

```
        // Test 3: simple carry
        apply_test(6'b001111, 6'b000001, 7'b0010000);  // 15 + 1 = 16

        // Test 4: long carry propagation
        apply_test(6'b111111, 6'b000001, 7'b1000000);  // 63 + 1 = 64

        // Test 5: maximum + maximum
        apply_test(6'b111111, 6'b111111, 7'b1111110);  // 63 + 63 = 126

        // Test 6: random middle value
        apply_test(6'b101010, 6'b010101, 7'b0111111);  // 42 + 21 = 63

        // Test 7: another random value
        apply_test(6'b100000, 6'b011111, 7'b0111111);  // 32 + 31 = 63

        #20;
        $finish;
    end

endmodule
```

B.2 tb_bist_adder6.v - 6-bit BISTed adder testbench

```
`timescale 1ps/1ps

module tb_bist_adder6;
    reg clk;
    reg reset;
    wire [3:0] lfsr_state;
    wire [5:0] a;
    wire [5:0] b;
    wire [6:0] sum;
    wire [6:0] signature;
    wire [4:0] pattern_count;
    wire done;

    integer PERIOD_PS;
    integer cycles;
    reg [1024*8:1] sdf_file;

    localparam [6:0] GOLDEN_SIGNATURE = 7'h4d;

    bist_adder6 dut (
        .clk(clk),
        .reset(reset),
        .lfsr_state(lfsr_state),
        .a(a),
        .b(b),
        .sum(sum),
        .signature(signature),
        .pattern_count(pattern_count),
        .done(done)
    );

    initial begin
        if (!$value$plusargs("PERIOD_PS=%d", PERIOD_PS)) PERIOD_PS = 10000; // 100 MHz
        clk = 1'b0;
```

```
        forever #(PERIOD_PS/2) clk = ~clk;
    end

    initial begin
`ifdef GATE
        if (!$value$plusargs("SDF=%s", sdf_file)) sdf_file = "build/syn6/bist_adder6_syn.sdf";
        $display("Annotating SDF: %0s", sdf_file);
        $sdf_annotate("build/syn6/bist_adder6_syn.sdf", tb_bist_adder6.dut);
`endif
    end

    initial begin
`ifdef FSDB
        $fsdbDumpfile("waves/bist6_gate.fsdb");
        $fsdbDumpvars(0, tb_bist_adder6);
`else
        $dumpfile("waves/bist6.vcd");
        $dumpvars(0, tb_bist_adder6);
`endif
    end

    initial begin
        reset = 1'b1;
        cycles = 0;
        repeat (2) @(negedge clk);
        reset = 1'b0;

        while ((done !== 1'b1) && (cycles < 100)) begin
            @(posedge clk);
            cycles = cycles + 1;
        end

        @(negedge clk);
        $display("HW3_RESULT N=6 PERIOD_PS=%0d signature=0x%0h expected=0x%0h done=%0b cycles=%0d",
                 PERIOD_PS, signature, GOLDEN_SIGNATURE, done, cycles);

        if ((done !== 1'b1) || (^signature === 1'bx) || (signature !== GOLDEN_SIGNATURE)) begin
            $display("HW3_FAIL");
            $finish(1);
        end else begin
            $display("HW3_PASS");
            $finish(0);
        end
    end
endmodule
```

B.3 tb_bist_adder12.v - 12-bit BISTed adder testbench

```
`timescale 1ps/1ps

module tb_bist_adder12;
    reg clk;
    reg reset;
    wire [3:0] lfsr_state;
    wire [11:0] a;
    wire [11:0] b;
    wire [12:0] sum;
    wire [12:0] signature;
    wire [4:0] pattern_count;
```

```
    wire done;

    integer PERIOD_PS;
    integer cycles;
    reg [1024*8:1] sdf_file;

    localparam [12:0] GOLDEN_SIGNATURE = 13'h458;

    bist_adder12 dut (
        .clk(clk),
        .reset(reset),
        .lfsr_state(lfsr_state),
        .a(a),
        .b(b),
        .sum(sum),
        .signature(signature),
        .pattern_count(pattern_count),
        .done(done)
    );

    initial begin
        if (!$value$plusargs("PERIOD_PS=%d", PERIOD_PS)) PERIOD_PS = 10000; // 100 MHz
        clk = 1'b0;
        forever #(PERIOD_PS/2) clk = ~clk;
    end

    initial begin
`ifdef GATE
        if (!$value$plusargs("SDF=%s", sdf_file)) sdf_file = "build/syn12/bist_adder12_syn.sdf";
        $display("Annotating SDF: %0s", sdf_file);
        $sdf_annotate("build/syn12/bist_adder12_syn.sdf", tb_bist_adder12.dut);
`endif
    end

    initial begin
`ifdef FSDB
        $fsdbDumpfile("waves/bist12_gate.fsdb");
        $fsdbDumpvars(0, tb_bist_adder12);
`else
        $dumpfile("waves/bist12.vcd");
        $dumpvars(0, tb_bist_adder12);
`endif
    end

    initial begin
        reset = 1'b1;
        cycles = 0;
        repeat (2) @(negedge clk);
        reset = 1'b0;

        while ((done !== 1'b1) && (cycles < 100)) begin
            @(posedge clk);
            cycles = cycles + 1;
        end

        @(negedge clk);
        $display("HW3_RESULT N=12 PERIOD_PS=%0d signature=0x%0h expected=0x%0h done=%0b cycles=%0d",
                 PERIOD_PS, signature, GOLDEN_SIGNATURE, done, cycles);

        if ((done !== 1'b1) || (^signature === 1'bx) || (signature !== GOLDEN_SIGNATURE)) begin
```

```
            $display("HW3_FAIL");
            $finish(1);
        end else begin
            $display("HW3_PASS");
            $finish(0);
        end
    end
endmodule
```

Appendix C. Speed Sweep Script

C.1 sweep_syn_sim.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Sweep maximum speed with SAME clock period for synthesis and gate simulation.
# Save fastest PASS FSDB and first FAIL FSDB to:
#   /home/course/ee625001/hw3_bist_adder/max
#
# Usage:
#   chmod +x scripts/sweep_syn_sim.sh
#   ./scripts/sweep_syn_sim.sh 6  10000 500 100
#   ./scripts/sweep_syn_sim.sh 12 10000 500 100

N=${1:-6}
START_PS=${2:-10000}
STOP_PS=${3:-500}
STEP_PS=${4:-100}

SAVE_ROOT="/home/course/ee625001/hw3_bist_adder/max"

if [[ "$N" == "6" ]]; then
    SYN=./scripts/syn6.sh
    SIM=./scripts/sim_gate6.sh
    NETLIST=build/syn6/bist_adder6_syn.v
    SDF=build/syn6/bist_adder6_syn.sdf
elif [[ "$N" == "12" ]]; then
    SYN=./scripts/syn12.sh
    SIM=./scripts/sim_gate12.sh
    NETLIST=build/syn12/bist_adder12_syn.v
    SDF=build/syn12/bist_adder12_syn.sdf
else
    echo "Error: N must be 6 or 12" >&2
    exit 2
fi

if [[ ! -x "$SYN" ]]; then
    echo "Error: $SYN not found or not executable."
    echo "Run: chmod +x scripts/syn${N}.sh"
    exit 2
fi

if [[ ! -x "$SIM" ]]; then
    echo "Error: $SIM not found or not executable."
```

```
    echo "Run: chmod +x scripts/sim_gate${N}.sh"
    exit 2
fi

mkdir -p logs waves
mkdir -p "build/best_synsim_N${N}"
mkdir -p "build/fail_synsim_N${N}"

PASS_FSDB_DIR="${SAVE_ROOT}/N${N}_fastest_pass"
FAIL_FSDB_DIR="${SAVE_ROOT}/N${N}_first_fail"

mkdir -p "$PASS_FSDB_DIR" "$FAIL_FSDB_DIR"

RESULT_CSV="logs/sweep_synsim_N${N}.csv"
echo "period_ps,clock_constraint_ns,frequency_mhz,syn_status,sim_status,result,signature_line" > "$RESULT_CSV"

BEST_PS=""
BEST_FREQ=""
FIRST_FAIL_SAVED=0

for ((P=${START_PS}; P>=${STOP_PS}; P-=${STEP_PS})); do
    CLK_NS=$(awk -v p="$P" 'BEGIN { printf "%.3f", p/1000.0 }')
    FREQ=$(awk -v p="$P" 'BEGIN { printf "%.3f", 1000000.0/p }')

    echo "============================================================"
    echo "[sweep-synsim] N=${N}, period=${P} ps, constraint=${CLK_NS} ns, freq=${FREQ} MHz"

    # Remove old FSDB before each simulation to avoid copying stale waveform.
    rm -f waves/*.fsdb 2>/dev/null || true

    # 1) Synthesis with the SAME clock period.
    set +e
    "$SYN" "$CLK_NS" > "logs/sweep_synsim_N${N}_${P}ps.syn.stdout" 2>&1
    SYN_STATUS=$?
    set -e

    # If synthesis exits non-zero but netlist and SDF exist, continue to simulation.
    # This avoids false SYN_FAIL caused by dc_shell return code.
    if [[ ! -f "$NETLIST" || ! -f "$SDF" ]]; then
        RESULT="SYN_FAIL"
        SIG_LINE=""
        echo "${P},${CLK_NS},${FREQ},${SYN_STATUS},NA,${RESULT},${SIG_LINE}" >> "$RESULT_CSV"
        echo "[result] ${RESULT} at ${P} ps"
        echo "  Missing netlist or SDF:"
        echo "  NETLIST=${NETLIST}"
        echo "  SDF=${SDF}"
        break
    fi

    # 2) Gate-level simulation with the SAME clock period.
    set +e
    "$SIM" "$P" > "logs/sweep_synsim_N${N}_${P}ps.sim.stdout" 2>&1
    SIM_STATUS=$?
    set -e

    SIM_LOG="logs/vcs_gate${N}_${P}ps.log"
    SIG_LINE=$(grep "HW3_RESULT" "$SIM_LOG" 2>/dev/null | tail -n 1 | tr ',' ';' || true)

    FSDB=$(find waves -maxdepth 1 -type f -name "*.fsdb" | head -n 1 || true)
```

```
if [[ $SIM_STATUS -eq 0 ]] \
   && grep -q "HW3_PASS" "$SIM_LOG" 2>/dev/null \
   && ! grep -Eiq "timing violation|setuphold|setup violation|hold violation|recovery violation|removal violation|width violation" "$SIM_LOG" 2>/dev/null; then

    RESULT="PASS"
    BEST_PS="$P"
    BEST_FREQ="$FREQ"

    # Save best netlist/SDF/log.
    cp -f "$NETLIST" "build/best_synsim_N${N}/" 2>/dev/null || true
    cp -f "$SDF"     "build/best_synsim_N${N}/" 2>/dev/null || true
    cp -f "$SIM_LOG" "build/best_synsim_N${N}/" 2>/dev/null || true
    cp -f "logs/sweep_synsim_N${N}_${P}ps.syn.stdout" "build/best_synsim_N${N}/" 2>/dev/null || true
    cp -f "logs/sweep_synsim_N${N}_${P}ps.sim.stdout" "build/best_synsim_N${N}/" 2>/dev/null || true

    # Keep only the current fastest PASS FSDB in max/N*_fastest_pass.
    if [[ -n "$FSDB" && -f "$FSDB" ]]; then
        rm -f "${PASS_FSDB_DIR}"/*.fsdb 2>/dev/null || true
        cp -f "$FSDB" "${PASS_FSDB_DIR}/fastest_pass_N${N}_${P}ps_${FREQ}MHz.fsdb"
        echo "  Saved PASS FSDB: ${PASS_FSDB_DIR}/fastest_pass_N${N}_${P}ps_${FREQ}MHz.fsdb"
    else
        echo "  Warning: no FSDB found for PASS period ${P} ps"
    fi

    echo "[result] PASS at ${P} ps (${FREQ} MHz)"

else
    RESULT="FAIL"

    # Save first FAIL FSDB only.
    if [[ "$FIRST_FAIL_SAVED" -eq 0 ]]; then
        if [[ -n "$FSDB" && -f "$FSDB" ]]; then
            rm -f "${FAIL_FSDB_DIR}"/*.fsdb 2>/dev/null || true
            cp -f "$FSDB" "${FAIL_FSDB_DIR}/first_fail_N${N}_${P}ps_${FREQ}MHz.fsdb"
            echo "  Saved FAIL FSDB: ${FAIL_FSDB_DIR}/first_fail_N${N}_${P}ps_${FREQ}MHz.fsdb"
        else
            echo "  Warning: no FSDB found for FAIL period ${P} ps"
        fi
        FIRST_FAIL_SAVED=1
    fi

    cp -f "$SIM_LOG" "build/fail_synsim_N${N}/" 2>/dev/null || true
    cp -f "logs/sweep_synsim_N${N}_${P}ps.syn.stdout" "build/fail_synsim_N${N}/" 2>/dev/null || true
    cp -f "logs/sweep_synsim_N${N}_${P}ps.sim.stdout" "build/fail_synsim_N${N}/" 2>/dev/null || true

    echo "[result] FAIL at ${P} ps"
    echo "  Check: logs/sweep_synsim_N${N}_${P}ps.syn.stdout"
    echo "  Check: logs/sweep_synsim_N${N}_${P}ps.sim.stdout"
    echo "  Check: ${SIM_LOG}"
fi

echo "${P},${CLK_NS},${FREQ},${SYN_STATUS},${SIM_STATUS},${RESULT},${SIG_LINE}" >> "$RESULT_CSV"

# Sweep from slow to fast. First FAIL after at least one PASS means maximum speed was crossed.
if [[ "$RESULT" == "FAIL" && -n "$BEST_PS" ]]; then
    break
fi
```

```
    if [[ "$RESULT" == "FAIL" && -z "$BEST_PS" ]]; then
        echo "No passing period found. Try a larger START_PS."
        break
    fi
done

echo "============================================================"
if [[ -n "$BEST_PS" ]]; then
    echo "Best passing period for N=${N}: ${BEST_PS} ps (${BEST_FREQ} MHz)"
    echo "Fastest PASS FSDB saved in: ${PASS_FSDB_DIR}"
    echo "First FAIL FSDB saved in: ${FAIL_FSDB_DIR}"
else
    echo "No passing period found in this range."
fi

echo "CSV saved to: ${RESULT_CSV}"
```